EKS Three-Tier Robot-Shop — Runbook

Cluster: **demo-cluster-three-tier-1** | Region: **ap-south-1** | Repo: `three-tier-architecture-robot-shop`

> ● **Highlighted values** are ones you supply or confirm yourself at run time — account IDs, VPC/security-group IDs, cluster/nodegroup names, IAM usernames, regions. Everything else can be copy-pasted as-is.

PART 1 — INSTALLATION (build from scratch)

0. Prerequisites

```
# kubectl
curl -LO "https://dl.k8s.io/release/$(curl -L -s https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubectl"
chmod +x kubectl && sudo mv kubectl /usr/local/bin/

# eksctl
curl --silent --location "https://github.com/eksctl-io/eksctl/releases/latest/download/eksctl_$(uname -s)_amd64.tar.gz"
sudo mv /tmp/eksctl /usr/local/bin

# AWS CLI v2
curl -fsSL https://awscli.amazonaws.com/v2/install.sh | bash

# Helm
curl -fsSL -o get_helm.sh https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3
chmod 700 get_helm.sh && ./get_helm.sh

# git (Oracle Linux / RHEL-based)
sudo dnf install -y git unzip

aws configure   # enter access key, secret key, region ap-south-1
```

1. Create the EKS cluster (control plane only, no nodegroup)

```
eksctl create cluster \
  --name demo-cluster-three-tier-1 \
  --region ap-south-1 \
  --without-nodegroup
```

2. Add a managed nodegroup

```
eksctl create nodegroup \
  --cluster demo-cluster-three-tier-1 \
  --name ng-c7i-flex-large \
  --node-type c7i-flex.large \
  --nodes 3 \
  --region ap-south-1
```

> Note: `t3.micro` free-tier nodes failed to create cleanly in this run — `c7i-flex.large` worked. Budget accordingly, it's not free-tier eligible.

3. Point kubectl at the new cluster

```
aws eks update-kubeconfig --region ap-south-1 --name demo-cluster-three-tier-1
kubectl get nodes
```

4. Associate IAM OIDC provider

```
eksctl utils associate-iam-oidc-provider --cluster demo-cluster-three-tier-1 --approve
```

5. AWS Load Balancer Controller

```
# IAM policy
curl -O https://raw.githubusercontent.com/kubernetes-sigs/aws-load-balancer-controller/v2.11.0/docs/install/iam_policy.
aws iam create-policy \
  --policy-name AWSLoadBalancerControllerIAMPolicy \
  --policy-document file://iam_policy.json

# Get your account ID first
aws sts get-caller-identity --query Account --output text

# IAM service account
eksctl create iamserviceaccount \
  --cluster demo-cluster-three-tier-1 \
  --namespace kube-system \
  --name aws-load-balancer-controller \
  --role-name AmazonEKSLoadBalancerControllerRole \
  --attach-policy-arn arn:aws:iam::<ACCOUNT_ID>:policy/AWSLoadBalancerControllerIAMPolicy \
  --approve

# Install via Helm
helm repo add eks https://aws.github.io/eks-charts
helm repo update eks
helm install aws-load-balancer-controller eks/aws-load-balancer-controller \
  -n kube-system \
  --set clusterName=demo-cluster-three-tier-1 \
  --set serviceAccount.create=false \
  --set serviceAccount.name=aws-load-balancer-controller \
  --set region=ap-south-1 \
  --set vpcId=<VPC_ID>

kubectl get deployment -n kube-system aws-load-balancer-controller
```

6. EBS CSI driver

```
eksctl create iamserviceaccount \
  --name ebs-csi-controller-sa \
  --namespace kube-system \
  --cluster demo-cluster-three-tier-1 \
  --role-name AmazonEKS_EBS_CSI_DriverRole \
  --role-only \
  --attach-policy-arn arn:aws:iam::aws:policy/service-role/AmazonEBSCSIDriverPolicy \
  --approve

eksctl create addon --name aws-ebs-csi-driver \
  --cluster demo-cluster-three-tier-1 \
  --service-account-role-arn arn:aws:iam::<ACCOUNT_ID>:role/AmazonEKS_EBS_CSI_DriverRole \
  --force
```

7. Deploy Robot Shop

```
git clone https://github.com/VenkataNaveenReddyYaparla/three-tier-architecture-robot-shop.git
cd three-tier-architecture-robot-shop/EKS/helm

kubectl create ns robot-shop
helm install robot-shop --namespace robot-shop .

kubectl get pods -n robot-shop
kubectl get svc web -n robot-shop
```

8. If LoadBalancer creation fails (account not enabled for ELB)

Fallback used previously — expose via NodePort and open the port range on the node security group:

```
kubectl patch svc web -n robot-shop -p '{"spec": {"type": "NodePort"}}'
```

```
aws ec2 authorize-security-group-ingress \
  --group-id <NODE_SG_ID> \
  --protocol tcp \
  --port 30000-32767 \
  --cidr 0.0.0.0/0 \
  --region ap-south-1
```

Then browse to `http://<node-external-ip>:<nodeport>`.

PART 2 — DESTROY (teardown, in order)

Order matters: delete app workloads and controllers before the nodegroup/cluster, or eksctl will hang trying to drain nodes that still have pods on them.

1. Uninstall the application

```
helm uninstall robot-shop -n robot-shop
kubectl delete ns robot-shop
```

2. Remove the LoadBalancer-related security group rule (manual add, won't self-clean)

```
aws ec2 revoke-security-group-ingress \
  --group-id <NODE_SG_ID> \
  --protocol tcp \
  --port 30000-32767 \
  --cidr 0.0.0.0/0 \
  --region ap-south-1
```

3. Uninstall the AWS Load Balancer Controller

```
helm uninstall aws-load-balancer-controller -n kube-system
```

4. Remove the EBS CSI addon

```
eksctl delete addon --name aws-ebs-csi-driver --cluster demo-cluster-three-tier-1 --region ap-south-1
```

5. Delete IAM service accounts (removes the CFN stacks + IAM roles)

```
eksctl delete iamserviceaccount \
  --cluster demo-cluster-three-tier-1 \
  --namespace kube-system \
  --name aws-load-balancer-controller \
  --region ap-south-1

eksctl delete iamserviceaccount \
  --cluster demo-cluster-three-tier-1 \
  --namespace kube-system \
  --name ebs-csi-controller-sa \
  --region ap-south-1
```

6. Delete the nodegroup(s)

```
eksctl delete nodegroup \
  --cluster demo-cluster-three-tier-1 \
  --name ng-c7i-flex-large \
  --region ap-south-1
```

Repeat for any other active nodegroup — check first with:

```
eksctl get nodegroup --cluster demo-cluster-three-tier-1 --region ap-south-1
```

7. Delete the cluster (also removes the OIDC provider and cluster stack)

```
eksctl delete cluster --name demo-cluster-three-tier-1 --region ap-south-1
```

8. Delete the standalone IAM policy

```
aws iam delete-policy --policy-arn arn:aws:iam::<ACCOUNT_ID>:policy/AWSLoadBalancerControllerIAMPolicy
```

9. Verify nothing billable is left

```
aws cloudformation list-stacks --region ap-south-1 --stack-status-filter CREATE_COMPLETE UPDATE_COMPLETE
aws ec2 describe-instances --region ap-south-1 --filters "Name=instance-state-name,Values=running"
aws elbv2 describe-load-balancers --region ap-south-1
aws ec2 describe-volumes --region ap-south-1 --filters "Name=status,Values=available"
```

10. Clean up local kubeconfig context (optional)

```
kubectl config delete-context arn:aws:eks:ap-south-1:<ACCOUNT_ID>:cluster/demo-cluster-three-tier-1
kubectl config delete-cluster demo-cluster-three-tier-1.ap-south-1.eksctl.io
```

11. Rotate the AWS access key

The key used in this session was pasted into plaintext logs — rotate it regardless of teardown:

```
aws iam create-access-key --user-name <IAM_USER>
aws iam delete-access-key --access-key-id <OLD_ACCESS_KEY_ID> --user-name <IAM_USER>
aws configure   # re-enter the new key
```

Known gotchas hit in this run (worth noting in your README)

- `t3.micro` nodegroup creation failed with a CFN error; had to delete the stack (termination protection was on) and retry with `c7i-flex.large`.
- `aws-load-balancer-controller` Helm install needs OIDC + IAM service account done first, or pods will crash-loop / never get the AWS permissions.
- Draining a nodegroup for delete can hang for several minutes if a pod (e.g. a StatefulSet like `redis-0`) isn't evictable — expected, just wait it out.
- ELB creation can fail with `OperationNotPermitted` on new/restricted AWS accounts — NodePort + manual SG rule is the workaround.